

シンプル計算機アプリ 解説資料

制作の流れ・ファイル関係・各ファイルの役割・React の使い方・制作手順 A~K (ヒント)

simple-calculator Vite + React 19 + TypeScript + Tailwind CSS v4 2026-09-12

先に結論: このアプリは **Next.js ではない**。1 ページの React アプリを **Vite** で動かしている。 `app/page.tsx` ・サーバーコンポーネント・ルーティングは未使用。 作り方は**完成コードではなくヒント**だけ書く（自分で実装する課題のため）。

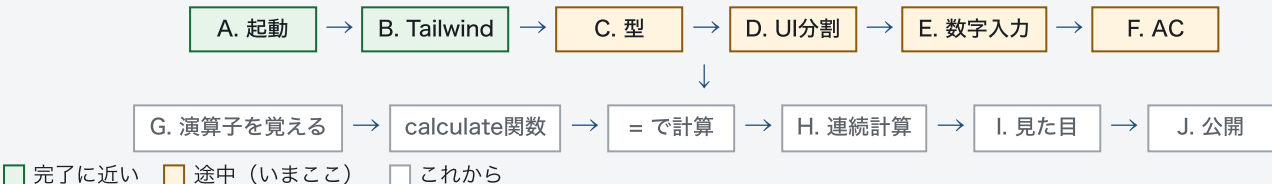
目次

1. 制作の流れ
2. ファイルの関係性
3. 各ファイルが何をしているか
4. React / Vite の機能の使い方
5. 制作手順 A~K (この通り進めれば完成する・ヒントのみ)

1. 制作の流れ

目標は iPhone 風電卓。見た目 → 数字入力 → 計算 → 連続計算 → 公開、の順に小さく進める。

図1. 制作ロードマップ (README の ToDo)



できていること	まだ途中
Vite起動 / Tailwind枠 / Display・Button分割 / 数字が表示が 変わる / ACで0	数字の連結・小数点・四則演算・連続計算・types.ts接続・iPhone風 色・GitHub Pages

図2. Git の流れ

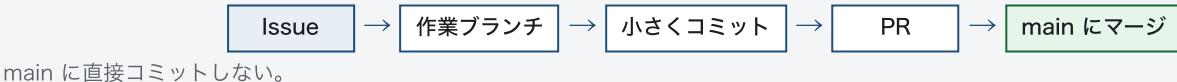


図3. 起動時

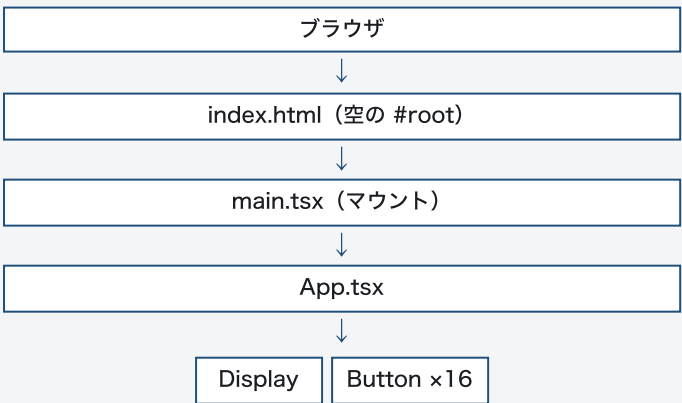
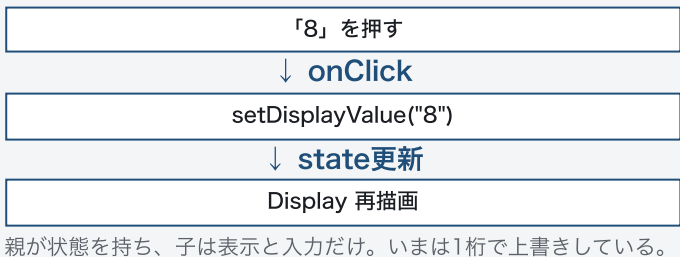
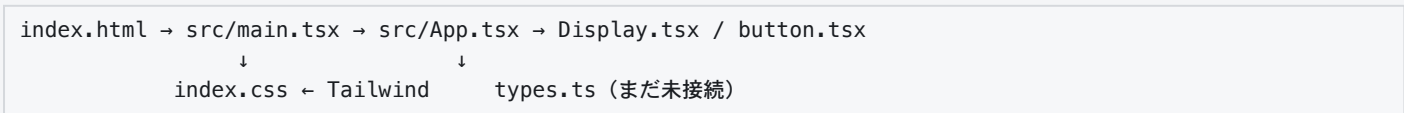


図4. 数字クリック（現在）



2. ファイルの関係性

図5. フォルダと import



Header は別ファイルではなく App.tsx 内。設定 (package.json / vite.config.ts / tsconfig) は画面に出ない。

親	子	Props	意味
App	Header	title, className	見出し「計算機」
App	Display	value, className	今の数字を表示
App	Button	label, className, onClick	文字・見た目・押した処理

3. 各ファイルが何をしているか

ファイル	役割	いまやっていること
index.html	ページの器	空の #root と main.tsx 読み込み
src/main.tsx	React の入口	createRoot + StrictMode で App を載せる
src/App.tsx	計算機の親	useState("0")、レイアウト、数字と AC の更新
Display.tsx	ディスプレイ	value を表示するだけ。計算しない
button.tsx	キー1個	label と任意の onClick
types.ts	型の置き場	Operator / Digit あり。状態型は空。未 import
index.css	スタイル入口	@import "tailwindcss" のみ
App.css	未使用	空。Vite 初期の名残
package.json	依存とスクリプト	npm run dev / build
vite.config.ts	Vite 設定	React プラグインと Tailwind プラグイン
tsconfig*	TS 設定	アプリ用 (DOM) と Vite 用 (Node) に分割
eslint.config.js	書き方チェック	Hooks ルール含む
public/	静的ファイル	favicon など変換しない
README.md	課題	A〜K の学習タスク

注意: App.tsx は ./components/Button を読み、実ファイルは button.tsx。Mac では動くが Linux (GitHub) では失敗しうる。名前を揃える。

4. React / Vite の機能の使い方

使っているのは React 本体と Vite。Next.js の App Router / SSR は未使用。

機能	どこ	このアプリでの意味
関数コンポーネント	App / Display / Button	画面の部品 = 関数。JSX を返す
JSX	各 .tsx	属性は className / onClick
Props	子コンポーネント	親から子へデータ。Display は value、Button は label
useState	App.tsx	表示文字列を覚える。更新で再描画
イベント	Button onClick	親の setter を呼ぶ
分割と再利用	components/	同じ Button を16回。label だけ変える
createRoot	main.tsx	HTML の root に React を載せる
StrictMode	main.tsx	開発中のチェック。本番の見たい目は変えない
export の種類	各ファイル	App は default。Display / Button は名前付き

```
const [displayValue, setDisplayValue] = useState("0");
<Display value={displayValue} />
<Button label="8" onClick={() => setDisplayValue("8")} />
```

やりたいこと	Next.js なら	このプロジェクト
開発サーバー	next dev	npm run dev (Vite)
ページ入口	app/page.tsx	index.html + main.tsx
保存して即反映	Fast Refresh	Vite HMR
本番ビルド	next build	npm run build

やりたいこと	Next.js なら	このプロジェクト
複数ページ / SSR	標準	使わない (1画面 SPA)

Tailwind: vite.config.ts のプラグイン + index.css の @import。grid grid-cols-4 が4列、bg-zinc-900 が暗い本体。

層	ファイル	仕事
表示	Display.tsx	文字列を出すだけ
入力	button.tsx	押されたことを親に伝えるだけ
状態	App.tsx (または Calculator.tsx)	今の数字・前の数字・演算子を useState で持つ
計算	utils/calculate.ts (未作成)	UI を知らない関数
型	types.ts	許す演算子と状態の形

5. 制作手順 A～K (README とおり・ヒントのみ)

この通り作成すれば制作できる。条件は「1タスク=1作業」「確認例をブラウザで満たしてから次へ」「コードは自分で書く」。完成関数は書かない。いまの位置は C～E (A・B は済)。main に直接コミットしない。

進め方とコミット

小さくコミットする目安: 1 Tailwind → 2 見た目UI → 3 数字と小数点 → 4 AC → 5 四則演算 (=まで) → 6 連続計算 → 7 デプロイ。

いま戻って直すこと (C～E を飛ばしている)

- 数字が上書き (setDisplayValue("8")) → E は末尾に足して 123 にする。
- CalculatorState = {} のまま、import していない → C。
- Calculator コンポーネントがない、色3種がない、並びが 8 7 9 → D。
- import が Button、ファイルが button.tsx → 名前を揃える。

A. 開発の始め方 済

- npm run dev で画面が出る。
- 作業ブランチで進める (例: feature/calc-page_hashimoto)。

ヒント不要。すでにできている。

B. Tailwind CSS 済

- index.css に Tailwind を読む記述がある。
- App.tsx の className で色・余白が変わっている。

ヒント: 効かないときは @import "tailwindcss" と vite.config.ts のプラグインを見比べる。公式は「Tailwind CSS v4 Vite」。

C. 型を先に決める いまここ

- src/types.ts を使う (作成済み)。演算子は + - × ÷ だけ。
- 状態の型: 画面の文字列 / 直前の数値 / 選んでいる演算子。
- ボタン見た目の型: 数字 / 演算子 / 機能 など。

空の {} では型にならない。値が無いときは null を型に含める。書いた型は使うファイルで importしないと効かない。

D. コンポーネントを分ける (見た目だけ。まだ計算しない) 途中

- Display: 表示する文字列だけ。右寄せ・大きな白い数字。
- Button: ラベル・色の種類・クリック関数を Props。色は3種 (暗い数字 / オレンジ演算子 / 明るい機能)。0 だけ幅2倍。
- Calculator: 黒い枠・角丸・幅固定。中に Display とボタン。
- 並び: AC ÷ / 7 8 9 × / 4 5 6 − / 1 2 3 + / 0 (横長) . =
- App は Calculator を中央。ページ背景は暗い。± と % は任意。

onClick は後回しでよい。色は親で全部書かず、種類 Props で切り替えると再利用しやすい。0 の横長は Tailwind の列スパンを調べる。今の 8→7→9 は iPhone と違う。

確認: 押さなくても電卓に見える。

E. 数字入力だけ動かす 途中

- 表示用 state の初期値は "0"。Display に渡す。
- 1~9 は末尾に付く。"0" のとき 5 は "5" ("05" ではない)。
- 0 連打で "00" にしない。・ は1回だけ。"0" + ・ は "0.1"。

いまは上書きなので、今の表示と押した文字を組み合わせる。分岐の目安は「今が0か」「小数点をすでに含むか」。演算子はこのステップでは無視。

確認: 1 → 2 → 3 で 123。AC はまだ動かなくてよい。

F. AC を動かす

- AC で表示が "0"。あとで演算の state を足したら、それらも初期値に戻す。

表示だけ戻して演算子を残すと、次の計算が壊れる。

G. 演算子を1つ覚える（計算はまだしない）

- previousValue（数値 or 空）と operator（Operator or 空）。
- 演算子を押したら、今の画面を前の値に保存。画面はまだ変えなくてよい。

確認は console.log で十分。文字列を数値にする方法を調べる。

確認: 1 のあと + で、前の値が 1、演算子が +。

G-2. 次の数字に切り替える

- waitingForOperand（boolean）。演算子で true。true のとき数字は上書き開始。出したら false。

この名前で検索すると実装の考え方が出やすい。フラグがないと 1 + 2 が 12 になる。

確認: 1 → + → 2 で画面が 1 から 2。まだ 3 にはならない。

G-3. 計算関数（UI と切り離す）

- src/utils/calculate.ts。calculate(a, b, operator) で四則の分岐。0除算は次。
- ボタンを押さず、コード上で直接呼んで確認。

この関数はボタンも state も知らない。if か switch。先に単体で検算する。

確認: 1 と 2 と + を渡すと 3。

G-4. = で1回だけ計算する

- 前の値・演算子・今の画面で calculate。結果を画面へ。0除算の表示を決める（Error か 0 など）。

材料が欠けていたら計算しない。Display は string なので、数値のまま渡せない。

確認: 7 + 8 = → 15

H. 連続計算（= を押さず演算子を続ける）

- すでに演算子がある状態で別の演算子 → 先に今の計算を実行してから、新しい演算子を覚える。

1 + 2 + の時点で画面に 3。演算子が「まだ無い」と「すでにある」で分岐する。

確認: 1 + 2 + 3 = → 6

H-2. = のあとの扱い

- = のあと数字 → 新しい計算。= のあと演算子 → 今の結果を前の値にして続ける。

「イコール直後か」も boolean で持つと分岐が書きやすい。詰まったら下の表を紙で埋める。

確認: 10 - 3 = → 7、続けて + 1 = → 8

紙に書く表（G～H で詰まったら）

1 + 2 + 3 =。空欄は自分で埋める（答えは書かない）。

押したキー	display	previousValue	operator	次の数字待ち？
（最初）	0	なし	なし	いいえ
1				
+				
2				
+（ここで一度計算）				
3				
=				

I. 見た目の仕上げ（任意だがおすすめ）

- 選んでいる演算子の色が変わる。桁が多いときは小さく、またははみ出さない。
- 未使用の Vite 初期ファイルを消す。README 末尾を自分のアプリ説明に。

必須のユーザーストーリーには無くても通せる。operator とボタンの label を比べると色を変えられる。

J. GitHub Pages へ出す

- Vite の base を調べ、リポジトリ名に合わせて設定。
- Actions か手動アップロード。URL で電卓が動くことを確認し、README に書く。

base は「サイトがルート以外に置かれるときの接頭辞」。合っていないと JS が読めず真っ白になりやすい。検索: Vite GitHub Pages base。

K. 提出前チェック

- iPhone風UI / 0～9 で数字 / 演算子のあと = で正しい結果 / 連続計算 1+2+3=6
- AC で 0 / デプロイ済み / TypeScript の型 / Display・Button・Calculator に分割
- GitHub 管理 / main 直コミットではなく 作業ブランチ → PR

調べるキーワード / つまづき

- React useState / Props の型 / TypeScript union type / Tailwind v4 Vite / 電卓 waitingForOperand
- 表示は string、計算は number。混ぜると "1"+"2"="12"。
- 演算子後にフラグがないと次の数字が繋がる。AC は演算の state も戻す。÷ 0 は無限大を出さない。

まとめ

1. README の A～K どれより進めれば、必須機能とユーザーストーリーは満たせる。
2. いまは C→D→E。E の「123」が通ってから F 以降へ。
3. 関係は index.html → main.tsx → App.tsx → Display / Button。状態は親が持つ。
4. React（コンポーネント・Props・useState・onClick）+ Vite。Next.js は未使用。

現行コード（App.tsx / Display.tsx / button.tsx / main.tsx / types.ts / README.md）に基づく。HTML をブラウザで開いて印刷→PDF でも再出力できる。